

Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Competitive Coding II_L8

VTU_Week 8_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an $M \times N$ matrix of characters, find the length of the longest path in the matrix starting from a given character.

In alphabetical order, all characters in the longest path should be increasing and consecutive. We are allowed to search the string in all eight possible directions, i.e., North, West, South, East, North-East, North-West, South-East, and South-West.

Use depth-first search (DFS) to solve this problem.

For example, consider the following matrix of characters:

The length of the longest path with consecutive characters starting from

character C is 6.

The longest path is:

Input Format

The first line of input is an integer n representing the row size.

The second line of input is an integer m representing the column size.

The following lines of input are the characters of the matrix.

The last line of the input is the character for which the length of the longest path is to be found.

Output Format

The output prints "The length of the longest path with consecutive characters starting from character <startChar> is <length>"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

5

d e h x b

a o g p e

d d c f d

e b e a s

c d y e n

c

Output: The length of the longest path with consecutive characters starting from character c is 6

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int N, M;
```

```
char matrix[MAX][MAX];
```

```
// Directions for 8 possible movements:
```

```
// North, South, East, West, and the 4 diagonals
```

```
int dx[] = {-1, 1, 0, 0, -1, -1, 1, 1};
```

```
int dy[] = {0, 0, 1, -1, -1, 1, -1, 1};
```

```
// Function to find the longest path using DFS
```

```
int findLongestPath(int r, int c, char targetChar) {
```

```
    int maxLength = 0;
```

```
    // Explore all 8 directions
```

```
    for (int i = 0; i < 8; i++) {
```

```
        int nextR = r + dx[i];
```

```
        int nextC = c + dy[i];
```

```
        // Check boundaries and if the next character is consecutive (Alphabetical order)
```

```
        if (nextR >= 0 && nextR < N && nextC >= 0 && nextC < M &&  
            matrix[nextR][nextC] == targetChar + 1) {
```

```
            int currentLength = findLongestPath(nextR, nextC, targetChar + 1);
```

```
            if (currentLength > maxLength) {
```

```
                maxLength = currentLength;
```

```
            }
```

```
        }
```

```
    }
```

```
    // Return current node (1) + the longest sub-path found
```

```
    return 1 + maxLength;
```

```
}
```

```
int main() {
```

```
    if (scanf("%d %d", &N, &M) != 2) return 0;
```

```
    // Read the matrix characters
```

```
    for (int i = 0; i < N; i++) {
```

```
        for (int j = 0; j < M; j++) {
```

```
            scanf(" %c", &matrix[i][j]);
```

```
        }
```

```

    }
    char startChar;
    scanf(" %c", &startChar);

    int overallMax = 0;

    // Find all occurrences of the starting character and start DFS from each
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            if (matrix[i][j] == startChar) {
                int res = findLongestPath(i, j, startChar);
                if (res > overallMax) {
                    overallMax = res;
                }
            }
        }
    }

    printf("The length of the longest path with consecutive characters starting
    from character %c is %d\n", startChar, overallMax);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Priya is developing a learning module to simulate breadth-first traversal of directed graphs. Given multiple test cases where each describes a directed graph with vertices and edges, write a program to perform BFS starting from vertex 0 and print the traversal order for each test case.

Example

Input:

```

1
5 4

```

0 1 0 2 0 3 2 4

Output:

0 1 2 3 4

Explanation:

For the given graph, the BFS is 0 1 2 3 4.

Input Format

The first line of input contains an integer T, denoting the number of test cases.

Then T test cases follow. Each test case consists of two lines.

The descriptions of the test cases are as follows:

1. The first line of each test case contains two space-separated integers, N and E, which denote the number of vertices and edges, respectively.
2. The second line of each test case contains E space-separated pairs u and v, denoting that there is an edge from u to v.

Output Format

For each test case, the output prints the BFS order of the graph vertices.

Each vertex should appear once in the order it is visited, separated by a space.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 2

5 4

0 1 0 2 0 3 2 4

3 2

0 1 0 2

Output: 0 1 2 3 4

0 1 2

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 10

void solve() {
    int N, E;
    if (scanf("%d %d", &N, &E) != 2) return;

    int adj[MAX][MAX] = {0};
    int visited[MAX] = {0};
    int queue[MAX];
    int front = 0, rear = 0;

    // Read E pairs of edges (u v)
    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        if (u < N && v < N) {
            adj[u][v] = 1; // Directed edge
        }
    }

    // BFS starting from vertex 0
    visited[0] = 1;
    queue[rear++] = 0;

    while (front < rear) {
        int current = queue[front++];
        printf("%d ", current);

        // Check neighbors in increasing order (0 to N-1)
        for (int i = 0; i < N; i++) {
            if (adj[current][i] == 1 && !visited[i]) {
                visited[i] = 1;
                queue[rear++] = i;
            }
        }
    }
    printf("\n");
}
```

```

}
int main() {
    int T;
    if (scanf("%d", &T) != 1) return 0;
    while (T--) {
        solve();
    }
    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Aayush is building a network verifier that must detect the presence of cycles in an undirected graph. Given a graph with a set number of vertices and edges, write a program to determine whether the graph contains at least one cycle using depth-first traversal.

Example

Input

N = 4, E = 4

Output

Yes

Explanation

The diagram clearly shows a cycle 0 to 2 to 1 to 0

Input

N = 4, E = 3

Output

No

Explanation

There is no cycle in the given graph.

Input Format

The first line of input consists of two integers, V and E, representing the number of vertices and edges separated by space, respectively.

The next E lines contain two space-separated integers u and v, describing an edge between vertices u and v.

Output Format

The output prints "Graph contains a cycle" if a cycle exists, otherwise prints "Graph does not contain a cycle".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 5

1 0

0 2

2 1

0 3

3 4

Output: Graph contains a cycle

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 10
```

```
// Function to check for cycle using DFS
```

```
int isCycleUtil(int v, int adj[MAX][MAX], int visited[MAX], int parent, int V) {  
    visited[v] = 1;
```

```
    for (int i = 0; i < V; i++) {  
        if (adj[v][i]) {
```

```

    // If neighbor is not visited, recurse for it
    if (!visited[i]) {
        if (isCycleUtil(i, adj, visited, v, V))
            return 1;
    }
    // If neighbor is visited and not the parent of current node, there's a cycle
    else if (i != parent) {
        return 1;
    }
}
}
return 0;
}

```

```

int main() {
    int V, E;
    int adj[MAX][MAX] = {0};
    int visited[MAX] = {0};

    if (scanf("%d %d", &V, &E) != 2) return 0;

```

```

    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        // Undirected graph
        adj[u][v] = 1;
        adj[v][u] = 1;
    }

```

```

    int cycleFound = 0;
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            if (isCycleUtil(i, adj, visited, -1, V)) {
                cycleFound = 1;
                break;
            }
        }
    }
}

```

```

    if (cycleFound) {
        printf("Graph contains a cycle\n");
    } else {

```

```
        printf("Graph does not contain a cycle\n");
    }
    return 0;
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Emma is developing a graph traversal feature for a social networking platform. Each user is represented by a lowercase English letter (a to z), and connections between users are represented as directed edges.

Emma wants to perform a Breadth-First Search (BFS) starting from a given user, visiting friends in lexicographical order whenever multiple connections are available.

Write a program to help Emma implement this lexicographically sorted BFS traversal.

Input:

N = 10, M = 10, S = 'a',

edges[[2] = { { 'a', 'y' }, { 'a', 'z' }, { 'a', 'p' }, { 'p', 'c' }, { 'p', 'b' }, { 'y', 'm' }, { 'y', 'l' }, { 'z', 'h' }, { 'z', 'g' }, { 'z', 'i' } }

Output:

a p y z b c l m g h i

Input Format

The first line contains an integer N, representing the number of nodes (users) in the graph.

The second line contains an integer M, representing the number of edges (friendships) in the graph.

The next M lines each contain two space-separated lowercase characters u and v, indicating a directed edge from user u to user v.

The last line contains an integer S, the starting node (user) for the BFS traversal.

Output Format

The output prints the users in the order they are visited during the BFS traversal, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10

10

a y

a z

a p

p c

p b

y m

y l

z h

z g

z i

a

Output: a p y z b c l m g h i

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define MAX 26
```

```
// Function to perform BFS
```

```
void bfs(int adj[MAX][MAX], int n, char start) {
```

```
    int startIdx = start - 'a';
```

```
    int visited[MAX] = {0};
```

```

int queue[MAX];
int front = 0, rear = 0;

visited[startIdx] = 1;
queue[rear++] = startIdx;

while (front < rear) {
    int u = queue[front++];
    printf("%c ", u + 'a');

    // By iterating from 0 to 25 (a to z),
    // we automatically visit neighbors in lexicographical order.
    for (int v = 0; v < MAX; v++) {
        if (adj[u][v] == 1 && !visited[v]) {
            visited[v] = 1;
            queue[rear++] = v;
        }
    }
}
printf("\n");
}

int main() {
    int n, m;
    int adj[MAX][MAX] = {0};

    // Read number of nodes and edges
    if (scanf("%d", &n) != 1) return 0;
    if (scanf("%d", &m) != 1) return 0;

    // Read M directed edges
    for (int i = 0; i < m; i++) {
        char u, v;
        scanf(" %c %c", &u, &v);
        adj[u - 'a'][v - 'a'] = 1;
    }

    char startNode;
    scanf(" %c", &startNode);
    bfs(adj, n, startNode);
}

```

```
} return 0;
```

Status : Correct

Marks : 10/10